# Week 2, Day 7 (Weekend): Revision, Mock Interview and REST API Design

**Prepared for Upendra Kumar Sharma | 3–4 hours | Main topics: Weeks 1 and 2 revision, timed mock interview on Java and Spring, REST API design for a complaint system**

## Today's Schedule

- **0:00–0:45** Revise with the rapid-recall sheets (section 1) and your flashcards.
- **0:45–1:45** Take the timed mock interview (section 2). Record yourself or explain to a friend.
- **1:45–2:30** REST API design exercise: design a complaint system API and explain it aloud (section 3).
- **2:30–3:00** DSA and SQL revision (section 4).
- **3:00–3:30** Score yourself, write your Week 2 review, and prepare for Week 3 (sections 5 and 6).

## 1. Week 2 Rapid-Recall Sheet

Read each line, cover the answer and say it from memory. Revisit the day guide for anything you cannot explain.

**Spring core and Boot**

- IoC means the container creates and wires objects. DI is the technique. Prefer **constructor injection**: final fields, easy testing, circular dependencies fail fast.
- Bean lifecycle: instantiate, inject, Aware callbacks, BeanPostProcessor before, @PostConstruct, afterPropertiesSet, BeanPostProcessor after (**proxies are created here**), ready, @PreDestroy.
- Singleton scope is one per container and must be stateless. A prototype inside a singleton needs ObjectProvider or a scoped proxy.
- Circular dependencies fail with constructor injection. Fix with redesign, events or @Lazy. Boot 2.6 and later prohibits them by default.
- AOP: aspect, pointcut, advice, join point. @Around must call proceed(). **Self-invocation bypasses the proxy.** Boot defaults to CGLIB proxies, which cannot override final classes or methods.
- Auto-configuration: loaded from the AutoConfiguration.imports file, guarded by conditions. @ConditionalOnMissingBean makes your beans win. Use --debug for the conditions report.

### Transactions, JPA and SQL

- @Transactional works through a proxy and a thread-bound connection. Rollback by default on **unchecked exceptions only**.
- Propagation: REQUIRED joins, REQUIRES_NEW suspends and starts a new transaction (needs a second connection), NESTED uses savepoints.
- Isolation: READ_COMMITTED stops dirty reads, REPEATABLE_READ stops non-repeatable reads, SERIALIZABLE stops phantoms. PostgreSQL defaults to READ COMMITTED.
- JPA entity states: transient, managed, detached, removed. Dirty checking writes changes at flush. merge returns a managed copy.
- **N+1:** 1 query plus N lazy loads. Fix with JOIN FETCH, @EntityGraph, DTO projections or batch fetching. Make associations LAZY.
- Do not paginate with a collection fetch join. Page the ids first.
- Indexes: B-tree default, leftmost prefix rule, covering indexes with INCLUDE, partial and expression indexes. **Foreign keys are not indexed automatically in PostgreSQL.**
- EXPLAIN ANALYZE: compare estimated and actual rows, find the slow node, look for sequential scans and sorts.
- **Keyset pagination** beats OFFSET on deep pages. Optimistic locking uses @Version, pessimistic uses FOR UPDATE, and SKIP LOCKED suits job queues.

### Security and REST

- Filter chain order: the JWT filter sits before UsernamePasswordAuthenticationFilter. 401 means not authenticated, 403 means not allowed.
- JWT is signed, not encrypted. Validate signature, exp, iss and aud. Keep access tokens short and rotate refresh tokens. Revocation is the hard part.
- OAuth2: Authorization Code with PKCE for apps with users, Client Credentials for services. OIDC adds the ID token and tells you who the user is.
- Method security uses @EnableMethodSecurity and @PreAuthorize, and also relies on proxies.
- REST: nouns for resources, correct status codes (201 with Location, 409 for conflicts), idempotency keys for POST, DTOs instead of entities, Problem Details for errors.
- Validation: @Valid on the body gives MethodArgumentNotValidException, @Validated on the class covers path and query parameters.

## 2. Timed Mock Interview (60 Minutes)

### How to run it

- Use a timer. Answer each question aloud in **90 seconds or less**, as you would in a real interview. Record yourself if you can.
- Score each answer: **2** = complete and confident, **1** = partly right or hesitant, **0** = could not answer.
- Mark every 0 and 1 and revisit the matching day guide afterwards.

**Part A: Java core (8 questions, about 12 minutes)**

1. Explain how HashMap works, including collisions and resizing.
2. What is the difference between ConcurrentHashMap in Java 7 and Java 8?
3. How does garbage collection work, and how would you choose between G1 and ZGC?
4. What does volatile guarantee, and what does it not?
5. How does ThreadPoolExecutor handle a new task, and how do you size a pool?
6. What are the four conditions for deadlock, and how do you prevent it?
7. How do you call three services in parallel with CompletableFuture and handle a timeout?
8. When should you use a parallel stream, and when should you not?

**Part B: Spring and Spring Boot (8 questions, about 12 minutes)**

9. Why is constructor injection preferred, and how do you break a circular dependency?
10. Walk through the bean lifecycle. Where are AOP proxies created?
11. Why does a @Transactional method called from the same class not run in a transaction?
12. Explain REQUIRED vs REQUIRES_NEW with a real use case.
13. What are the rollback rules, and what is the rollback-only trap?
14. How does Spring Boot auto-configuration work, and how do you debug it?
15. How would you secure a stateless REST API with JWT?
16. How do you handle validation and errors in a consistent way?

**Part C: JPA, Hibernate and SQL (6 questions, about 10 minutes)**

17. What is the N+1 problem, how do you detect it and how do you fix it?
18. What are the four entity states? What is the difference between persist and merge?
19. Lazy vs eager loading, and what causes LazyInitializationException?
20. How do you decide which columns to index, and why might an index not be used?
21. How do you read an EXPLAIN ANALYZE plan?
22. Optimistic vs pessimistic locking: how does each work and when would you use each?

**Part D: Coding and SQL (about 15 minutes)**

23. Solve one of these on paper or in an editor, thinking aloud: Search in Rotated Sorted Array, Daily Temperatures, Merge Intervals or Linked List Cycle.
24. Write SQL for the top three distinct salaries per department.

**Part E: Behavioral (about 10 minutes)**

25. Tell me about a production issue you debugged. What was the symptom, root cause, fix and result?
26. Tell me about a time you improved performance, with before and after numbers.
27. Describe the most complex integration you built, and the trade-offs you made.

For questions 25 to 27, use the STAR structure (situation, task, action, result) and only describe work you actually did.

**Scoring (questions 1 to 22, maximum 44)**

- 36–44: strong. Fix the few weak spots and move on.
- 28–35: decent. Reread the sections for every 0 and 1.
- Below 28: spend extra time on the weak topics before the Week 3 mock.

## 3. REST API Design Exercise: Complaint Management System

**The prompt (say this out loud as an interviewer would):** "Design the REST API for a complaint management system. Citizens or registered organizations submit complaints with documents, officers review and resolve them, and admins manage users. Walk me through your resources, endpoints, security, error handling and how you would handle scale and concurrency."

Treat this as an exercise. Where it overlaps with systems you built, use your real experience, and keep your claims honest.

**Step 1: Clarify requirements (30 seconds aloud)**

- Who are the users? Complainant, officer, admin, and perhaps a read-only auditor.
- What is the lifecycle? How many complaints per day, and how large are documents?
- Must we keep an audit trail? Do users need notifications? Is there an SLA for resolution?

**Step 2: Resources and endpoints (all under /api/v1)**

- **POST /complaints**: submit a complaint. Returns 201 with a Location header. Accepts an Idempotency-Key header. Role: complainant.
- **GET /complaints**: list with filters (status, entityId, dateFrom, dateTo), sorting and pagination. Complainants see only their own, officers see assigned or department complaints. Returns 200.

- **GET /complaints/{id}**: full details. Returns 200, 404, or 403 if not allowed.
- **PUT or PATCH /complaints/{id}**: edit a draft. Requires If-Match with the version, otherwise 412 (or 409 on conflict).
- **POST /complaints/{id}/transitions** with a body such as {"action": "ASSIGN", "assigneeId": 7}: moves the complaint through its lifecycle. Returns 200, or 409 if the transition is not valid from the current state. Role depends on the action.
- **POST /complaints/{id}/documents** (multipart): upload a document. Returns 201. **GET /complaints/{id}/documents/{docId}**: download, authorized per request.
- **GET /complaints/{id}/history**: audit trail of status changes. Role: officer, admin, auditor.
- **POST /complaints/{id}/comments** and **GET /complaints/{id}/comments**: internal and public notes.
- **DELETE /complaints/{id}**: admin only, and consider soft delete for retention rules.
- **Auth endpoints:** POST /auth/login and POST /auth/refresh, or delegate to an identity provider with OAuth2 and OIDC.

**Step 3: Lifecycle (state machine)**

- DRAFT, then SUBMITTED, then UNDER_REVIEW, then ASSIGNED, then RESOLVED, then CLOSED. Side paths: REJECTED, and REOPENED from RESOLVED or CLOSED.
- Keep the allowed transitions in one place (an enum or a state machine class), validate every transition on the server, and write each change to the history table in the same transaction.
- Using a transitions sub-resource instead of a free status field prevents invalid jumps and makes each action auditable and separately authorized.

**Step 4: Data model (short)**

- **complaint:** id, reference_no (unique), title, description, status, entity_id, complainant_id, assigned_to, created_at, updated_at, **version**.
- **complaint_document:** id, complaint_id, file name, content type, size, storage key, checksum, uploaded_by.
- **complaint_status_history:** id, complaint_id, from_status, to_status, actor, timestamp, remark.
- **Indexes:** (entity_id, status, created_at DESC) for officer lists, (assigned_to, status) for work queues, a unique index on reference_no, and indexes on foreign keys.

**Step 5: Cross-cutting design decisions**

- **Security:** JWT or OAuth2 resource server. URL-level role rules, method-level @PreAuthorize, plus **data-level checks** (a complainant can only touch their own complaints). Return 404 instead of 403 when you do not want to reveal that a record exists.

- **Validation:** Bean Validation on request DTOs, state checks in the service, and database constraints as a last line.
- **Errors:** RFC 7807 Problem Details with a trace id. 400 for validation, 401, 403, 404, 409 for invalid transitions and duplicates, 413 for oversized files, 429 for rate limiting.
- **Concurrency:** optimistic locking with @Version, surfaced as a version in the response and If-Match on updates. Two officers assigning the same complaint then produce a clean 409.
- **Idempotency:** the Idempotency-Key on POST /complaints stops duplicate complaints from double-clicks and retries. Store the key with the created resource id and a unique constraint.
- **Files:** enforce size and type limits, scan for malware, store the file in object storage or a document management system, and keep only metadata and a storage key in the database. Stream downloads through an authorized endpoint, or use short-lived signed URLs.
- **Pagination:** page and size is fine for admin screens. Use keyset pagination for large or fast-growing lists.
- **Notifications and integrations:** publish events after commit (the outbox pattern, coming in Week 3) so email, SMS and downstream systems stay consistent with the database.
- **Observability and operations:** correlation ids in logs, metrics on request latency and error rate, health endpoints, rate limiting, and an audit log.
- **Evolution:** URL versioning, additive changes only, and OpenAPI documentation.

**Step 6: Explain trade-offs aloud**

- Why a transitions endpoint instead of PATCH status? (Control, audit, per-action authorization.)
- Why optimistic rather than pessimistic locking here? (Conflicts are rare, and users should not hold locks while thinking.)
- Why keep files out of the database? (Size, backup cost, streaming, and scalable storage.)
- What would change at 100 times the traffic? (Read replicas, caching of reference data, an async pipeline for documents, keyset pagination, rate limits.)

Time yourself. A good answer fits in **10 to 12 minutes**, covering requirements, endpoints, lifecycle, security, errors, concurrency and scale.

## 4. DSA and SQL Revision

**Redo from scratch with a 20-minute limit each**

- Search in Rotated Sorted Array (binary search).
- Daily Temperatures (monotonic stack).
- Merge Intervals or Non-overlapping Intervals (sort, then scan or greedy).
- Linked List Cycle, then the cycle start follow-up (fast and slow pointers).
- The top three distinct salaries per department in SQL.

**Week 2 pattern cheat sheet**

- **Binary search:** use mid = lo + (hi - lo) / 2. For exact matches use lo <= hi. For boundaries use lo < hi with hi = mid. In a rotated array, one half is always sorted.
- **Stack:** matching pairs (Valid Parentheses), and **monotonic stack** for the next greater element (Daily Temperatures). Track extra state with each element (Min Stack).
- **Intervals:** sort first. Sort by **start** to merge, by **end** for greedy selection. Decide whether touching intervals overlap.
- **Linked list:** a **dummy head** removes edge cases. Use prev, curr and next for reversal, and **fast and slow pointers** for cycles and midpoints.
- **SQL:** window functions with PARTITION BY for per-group ranking, DENSE_RANK for top-N distinct values, and NOT EXISTS for anti-joins.

## 5. Week 2 Review and Reflection

Write answers in your notes:

- What was my mock score, and which questions scored 0 or 1?
- Which Spring behaviors did I only understand after trying them in code?
- Did I finish the secured API from Day 6? If not, what is left?
- Which DSA problems from this week do I still not trust myself on?
- What will I change in my daily routine for Week 3?

## 6. Week 3 Preview: Microservices Patterns

- **Day 1:** microservice principles, decomposition, API Gateway, service discovery, config server. DSA: Remove Nth Node From End, Reorder List, LRU Cache.
- **Day 2:** inter-service communication (REST, Feign, gRPC basics). DSA: Implement Queue using Stacks, Sliding Window Maximum.
- **Day 3:** Resilience4j (circuit breaker, retry, bulkhead, rate limiter). DSA: tree traversals.
- **Day 4:** saga, outbox, CQRS, idempotency. DSA: BST problems.
- **Day 5:** Kafka fundamentals. DSA: tree problems.

- **Day 6:** start the mini project with three services, Docker Compose and JWT at the gateway.
- **Day 7:** system design introduction and a URL shortener.

**Before Week 3**

- Make sure your Day 6 secured API is committed to GitHub. It becomes the first service of the mini project.
- Install Docker Desktop (or Docker Engine) and confirm that docker compose works.
- If your application hunt starts in Week 6, begin collecting job descriptions now, so you can see which skills appear most often.

## 7. End-of-Day Checklist

- [ ] I reviewed both rapid-recall sheets and my flashcards.
- [ ] I completed the timed mock and recorded my score and weak topics.
- [ ] I explained the complaint API design aloud in 10 to 12 minutes.
- [ ] I redid five DSA and SQL problems from this week.
- [ ] I wrote my Week 2 review notes.
- [ ] My secured API is on GitHub, and Docker works on my machine.

**Tomorrow (Week 3, Day 1):** microservice principles, service decomposition, API Gateway, service discovery and config server, plus linked list problems.